

Práctica GitHub – Sergio Mesa

Alumno: Sergio Mesa

Curso: 2ASIR

Fecha: 24/09/2026

Objetivo: En esta práctica hemos configurado GitHub para usar SSH, creado y clonado repositorios, subido archivos y utilizado sintaxis Markdown en un README.

1. Configuración de GitHub

1.1. Creación de la cuenta

Primero he creado una cuenta en GitHub accediendo a la siguiente dirección:

<https://github.com>

Una vez creada la cuenta de GitHub vamos a continuar generando y copiando nuestra clave SSH en GitHub

1.2. Generación de la clave H

Para poder conectarnos a GitHub mediante SSH, he comprobado primero si ya tenía una clave pública RSA creada:

```
ls -l ~/.ssh/
```

En caso de no tener los archivos `id_rsa` e `id_rsa.pub`, hay que generar una clave nueva con el siguiente comando:

```
ssh-keygen -t rsa -b 4096 -C "sergiomesamejias@gmail.com"
```

En mi caso, yo ya tengo los archivos de mi clave publica/privada entonces no voy a generar otra.

1.3. Copia de la clave pública

Después, he mostrado el contenido de mi clave pública con este comando:

```
cat ~/.ssh/id_rsa.pub
```

He copiado la línea completa mostrada en pantalla, que empieza por `ssh-rsa` y termina normalmente con un comentario, como mi correo electrónico.

Salida del comando `cat ~/.ssh/id_rsa.pub`.

```
sergio@Sergio-PC:~$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQDA7298jiVrq8NT6opE7Y0a+Rxkeytub7hBuTzMQOI1RWKVq3TTI3vmNMj0wvFABVuK
SEuiRngStWpX5MptUyhUqTTrIX/wLP3M4g5jmw7b/YxulY4L5PSwe9ntrnz1wKXm0sK0sbBv8oM66foyuruAtiwjBRZCcEMliigzFajG
lWB1H4pXRVF0FvAgRQMLHN9v40rs8tZdL7cWXTxy7Ni1MuNZ5s4Z3BvbQrN2iu3dJ6eTmTHPAytA+d3msTQvA2Qxjy60qvMy4unwizqM
Jjj63SG7HE5ul4K7v6A6piWkWoCpl/yhB37pNigKUrKY38Y8hjrLc6UQ652YyXyzlNmp5h+Wb+dq+QponjGBF5JoCRnoKwvAMcF0KLF
QxDhfNvnK96P0Lu/HMJU22w3j6qJ09csEYVoPrueB1wP+37hKMqBYfcCfhR5su72eGbc6X7xwrP4IAoKVYydiG01a0CQHdquXNu16Xk
NsSTRme0tha82wxQcsBIjjsVDxFlwTmYKznVZXCFACdpYlcJUXWBk4zprl4dzh9Q5mXpCIMAjdphJs6zw0nz+hy6fi871BClyjreJX
+CwAPkJ6hS0zo4hhNQ0XLtCf9k1ET+JYs+BzMGnx7SF7QSAa5PXzf8XphqA5JRo4dY/dMgy7l8GVFW1XBe0lKgXVepnFkx5FB0== ser
gio@Sergio-PC
```

1.4. Añadir la clave SSH a GitHub

Desde mi perfil de GitHub he seguido estos pasos:

1. He pulsado sobre mi foto de perfil y he entrado en **Settings**.
2. He accedido al apartado **SSH and GPG keys**.
3. He pulsado en **New SSH key**.
4. He escrito un nombre identificativo para mi ordenador.
5. He pegado la clave pública RSA que había copiado anteriormente.
6. Finalmente, hemos guardado la clave pulsando en **Add SSH key**.

Clave SSH añadida a mi perfil de GitHub.

1.5. Comprobar conexión SSH

He comprobado que GitHub reconoce correctamente mi clave usando el siguiente comando:

```
ssh -T git@github.com
```

La primera vez es posible que aparezca una pregunta para confirmar la huella del servidor. En ese caso, he escrito `yes`.

El mensaje esperado debe indicar que la autenticación se ha realizado correctamente, aunque GitHub no proporciona acceso a una shell remota.

Comprobación de la conexión SSH con GitHub.



SergioMM (Sergio852)

Your personal account

Public profile

Account

Appearance

Accessibility

Notifications

Access

Billing and licensing

Emails

Password and authentication

Sessions

SSH and GPG keys

Credentials

Organizations

Enterprises

Moderation

Add new SSH Key

Title

PORTATIL APPWEB

Key type

Authentication Key

Key

```
ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQDA7298jiVrq8NT6opE7Y0a+Rxkeytub7hBuTzMO
Ql1RWKVq3TTl3vmNMj0wvFABVuKSEuiRngStWpX5MptUyhUqTTrIX/wLP3M4g5jmw7b/
YxulY4L5PSwe9ntrnz1wKXmOsKOsBv8oM66foyuruAtiwjBRZCcEM1iigzFajGiWB1H4pXRV
F0FvAgRQMIHN9v4Ors8tZdl7cWXTxy7Ni1MuNZ5s4Z3BvbQrN2iu3dJ6eTmTHPAytA+d3ms
TQvA2Qxjy60qvMy4unwizqMJj63SG7HE5ul4K7v6A6piWkWoCpl/
yhB37pNigKURkY38Y8hjrIc6UQ652YyXyzlNmp5h+Wb+dg+QponjGBF5JoCRnoKwwwAMcFO
KLFQxDhfNvnK96POLu/
HmJuN22w3j6qJ09csEYVoPrueB1wP+37hKMqBYfcCfhR5su72eGBc6X7xwrP4IAoKVYydiG
```

Add SSH key

2. Primer repositorio: prueba_sergio_mesa

2.1. Creación del repositorio remoto

En GitHub he creado un repositorio nuevo con estos datos:

Campo	Valor introducido
Nombre del repositorio	prueba_sergio_mesa
Descripción	Repositorio de prueba 2ASIR
Inicialización	README habilitado
Visibilidad	Pública

Formulario de creación del repositorio prueba_sergio_mesa .

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner *



Repository name *

/ prueba_sergio_mesa

✔ prueba_sergio_mesa is available.

Great repository names are short and memorable. How about [super-enigma](#)?

Description

Repositorio de prueba 2ASIR

27 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

On ☒

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

Repositorio recién creado en GitHub.

 **prueba_sergio_mesa** Public

 Pin  Watch **0**

 main   Go to file + <> Code

 **Sergio852** Initial commit d758b4d · now

 README.md

Initial commit

now

README 

prueba_sergio_mesa

Repositorio de prueba 2ASIR

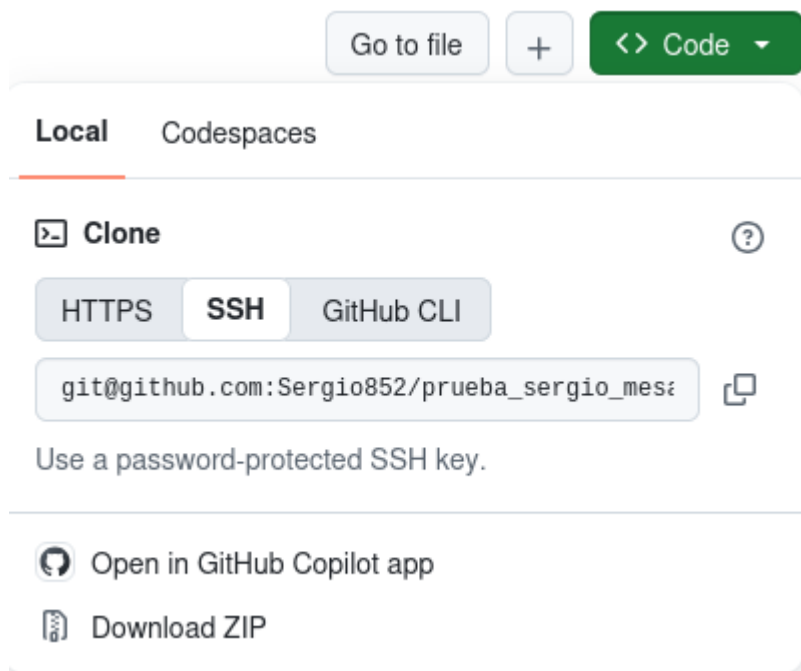
2.2. Copiar la URL SSH

Una vez creado el repositorio, he pulsado el botón verde **Code**, he seleccionado la pestaña **SSH** y he copiado la URL SSH. La URL tiene un formato similar a este:

```
git@github.com:USUARIO_GITHUB/prueba_sergio_mesa.git
```

Importante: He utilizado la URL SSH y no la URL HTTPS.

URL SSH del repositorio copiada desde GitHub.



2.3. Clonar el repositorio

He creado una carpeta para guardar mis prácticas de GitHub y he clonado el repositorio remoto mediante SSH:

```
cd ~  
mkdir -p github_practicas  
cd github_practicas  
git clone git@github.com:USUARIO_GITHUB/prueba_sergio_mesa.git
```

A continuación, he entrado en la carpeta clonada:

```
cd prueba_sergio_mesa
```

Clonado del repositorio mediante SSH.

```
sergio@Sergio-PC:~$ cd ~  
sergio@Sergio-PC:~$ mkdir -p github_practicas  
sergio@Sergio-PC:~$ cd github_practicas/  
sergio@Sergio-PC:~/github_practicas$ git clone git@github.com:sergio852/prueba_sergio_mesa.git  
Clonando en 'prueba_sergio_mesa'...  
remote: Enumerating objects: 3, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
Recibiendo objetos: 100% (3/3), listo.  
sergio@Sergio-PC:~/github_practicas$
```

2.4. Comprobación de la URL remota

Para comprobar que hemos configurado el repositorio utilizando SSH, he mostrado el contenido del archivo `.git/config`:

```
cat .git/config
```

En la sección `[remote "origin"]` debe aparecer una línea similar a la siguiente:

```
url = git@github.com:sergio852/prueba_sergio_mesa.git
```

También he comprobado el repositorio remoto con:

```
git remote -v
```

Contenido de `.git/config` y salida de `git remote -v`.

```
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$ cat .git/config
[core]
  repositoryformatversion = 0
  filemode = true
  bare = false
  logallrefupdates = true
[remote "origin"]
  url = git@github.com:sergio852/prueba_sergio_mesa.git
  fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
  remote = origin
  merge = refs/heads/main
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$ git remote -v
origin  git@github.com:sergio852/prueba_sergio_mesa.git (fetch)
origin  git@github.com:sergio852/prueba_sergio_mesa.git (push)
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$
```

2.5. Comprobación del README

El repositorio se inicializó con un archivo llamado `README.md`. He comprobado que existe y he mostrado su contenido:

```
ls -la
cat README.md
```

Este archivo sirve para describir el proyecto y se muestra automáticamente en la página principal del repositorio de GitHub.

Archivo `README.md` dentro del repositorio local.

```
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$ ls -la
cat README.md
total 16
drwxrwxr-x 3 sergio sergio 4096 sep 24 12:17 .
drwxrwxr-x 3 sergio sergio 4096 sep 24 12:17 ..
drwxrwxr-x 8 sergio sergio 4096 sep 24 12:17 .git
-rw-rw-r-- 1 sergio sergio  49 sep 24 12:17 README.md
# prueba_sergio_mesa
Repositorio de prueba 2ASIR
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$
```

3. Creación y modificación de archivos

3.1. Crear carpetas, subcarpetas y archivos

Dentro del repositorio `prueba_sergio_mesa`, he creado varias carpetas y archivos para comprobar el flujo de trabajo con Git:

```
mkdir -p documentos/apuntes
mkdir -p codigo

echo "Estos son mis apuntes de Git y GitHub." > documentos/apuntes/git.txt
echo "print('Hola desde Python')" > codigo/ejemplo.py
echo "# Documento adicional" > documento_adicional.md
```

Después, he comprobado la estructura creada:

```
tree github_practicas/
```

Creación de carpetas, subcarpetas y archivos.

```
sergio@Sergio-PC:~$ tree github_practicas/
github_practicas/
├── prueba_sergio_mesa
│   ├── codigo
│   │   └── ejemplo.py
│   ├── documento_adicional.md
│   ├── documentos
│   │   └── apuntes
│   │       └── git.txt
│   └── README.md
5 directories, 4 files
sergio@Sergio-PC:~$
```

3.2. Consultar el estado del repositorio

Antes de guardar los cambios, he ejecutado:

```
git status
```

Con este comando podemos ver que Git detecta los archivos nuevos como archivos no rastreados (*untracked files*).

Estado del repositorio antes de añadir los archivos.

```
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Archivos sin seguimiento:
  (usa "git add <archivo>..." para incluirlo a lo que será confirmado)
    codigo/
    documento_adicional.md
    documentos/

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa "git add" para hacerles seguimiento)
```

3.3. Añadir archivos al área de preparación

He añadido todos los cambios al área de preparación o *staging area*:

```
git add .
```

Después, hemos comprobado de nuevo el estado:

```
git status
```

Ahora los archivos deben aparecer preparados para el siguiente commit.

Archivos añadidos con `git add .`

```
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$ git add .
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$ git status
En la rama main
Tu rama está actualizada con 'origin/main'.

Cambios a ser confirmados:
  (usa "git restore --staged <archivo>..." para sacar del área de stage)
    nuevos archivos: codigo/ejemplo.py
    nuevos archivos: documento_adicional.md
    nuevos archivos: documentos/apuntes/git.txt

sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$
```

3.4. Crear un commit local

He guardado los cambios en el historial local del repositorio con un commit:

```
git commit -m "Añadidos archivos y estructura inicial"
```

Podemos consultar el historial de commits mediante:

```
git log --oneline
```

Creación del commit y visualización del historial.

```
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$ git commit -m "Añadidos archivos y estructura inicial"
[main afe8417] Añadidos archivos y estructura inicial
 3 files changed, 3 insertions(+)
 create mode 100644 codigo/ejemplo.py
 create mode 100644 documento_adicional.md
 create mode 100644 documentos/apuntes/git.txt
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$ git log --oneline
afe8417 (HEAD -> main) Añadidos archivos y estructura inicial
d758b4d (origin/main, origin/HEAD) Initial commit
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$
```

3.5. Subir los cambios a GitHub

Finalmente, he enviado el commit al repositorio remoto de GitHub:

```
git push origin main
```

Si la rama principal de mi repositorio se llama `master`, debo utilizar este comando en su lugar:

```
git push origin master
```

Tras actualizar la página de GitHub, hemos comprobado que los archivos y carpetas creados aparecen en el repositorio remoto.

Salida de `git push`.

```
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$ git push origin main
Enumerando objetos: 9, listo.
Contando objetos: 100% (9/9), listo.
Compresión delta usando hasta 10 hilos
Comprimiendo objetos: 100% (2/2), listo.
Escribiendo objetos: 100% (8/8), 605 bytes | 605.00 KiB/s, listo.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: This repository moved. Please use the new location:
remote:  git@github.com:Sergio852/prueba_sergio_mesa.git
To github.com:sergio852/prueba_sergio_mesa.git
    d758b4d..afe8417  main -> main
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$
```

Archivos visibles en el repositorio de GitHub.

 **prueba_sergio_mesa** Public Pin Watch 0

main Go to file + Code

 **Sergio852** Añadidos archivos y estructura inicial afe8417 · 1 minute ago

 codigo	Añadidos archivos y estructura inicial	1 minute ago
 documentos/apuntes	Añadidos archivos y estructura inicial	1 minute ago
 README.md	Initial commit	18 minutes ago
 documento_adicional.md	Añadidos archivos y estructura inicial	1 minute ago

4. Segundo repositorio: práctica 1

4.1. Crear el repositorio remoto

Para almacenar en GitHub el repositorio local creado durante la práctica 1, he creado otro repositorio en GitHub con el nombre:

```
practicalsergiomesa
```

He dejado sin marcar la opción de inicializar el repositorio con README, `.gitignore` o licencia, ya que mi repositorio local de la práctica 1 ya contiene archivos y posiblemente commits previos.

4.2. Asociar el repositorio local al remoto

He entrado en la carpeta donde se encuentra mi repositorio local de la práctica 1:

```
cd ~/sergio_mesa_mejias/
```

He comprobado el estado y los repositorios remotos configurados:

```
git status
git remote -v
```

Si no había ningún remoto llamado `origin`, lo he añadido con la URL SSH del repositorio recién creado:

```
git remote add origin git@github.com:Sergio852/practicalsergiomesa.git
```

Configuración del remoto `origin` para la práctica 1.

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ git remote add origin git@github.com:sergio852/practicalsergiomesa.git
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

4.3. Subir la práctica 1 a GitHub

He comprobado el nombre de mi rama actual:

```
git branch --show-current
```

Si quiero que la rama principal se llame `main`, puedo renombrarla:

```
git branch -M main
```

Después, he subido el repositorio local a GitHub:

```
git push -u origin master
```

La opción `-u` asocia la rama local con la rama remota. De este modo, en los próximos cambios bastará normalmente con ejecutar `git push`.

Subida de la práctica 1 al segundo repositorio.

```
sergio@Sergio-PC:~/sergio_mesa_mejias$ git push -u origin master
Enumerando objetos: 15, listo.
Contando objetos: 100% (15/15), listo.
Compresión delta usando hasta 10 hilos
Comprimiendo objetos: 100% (14/14), listo.
Escribiendo objetos: 100% (15/15), 1.66 KiB | 1.66 MiB/s, listo.
Total 15 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), done.
remote: This repository moved. Please use the new location:
remote:   git@github.com:Sergio852/practicalsergiomesa.git
To github.com:sergio852/practicalsergiomesa.git
 * [new branch]      master -> master
rama 'master' configurada para rastrear 'origin/master'.
sergio@Sergio-PC:~/sergio_mesa_mejias$
```

Contenido de la práctica 1 visible en GitHub.

 **practica1sergiomesa** Public Pin Watch 0

master Go to file + Code

 **Sergio852** Revert "Recuperar index.html" 44bd548 · 2 days ago

 .gitignore	Configurar archivos ignorados	2 days ago
 index.html	Revert "Recuperar index.html"	2 days ago
 styles.css	Renombrar estilos.css a styles.css	2 days ago

5. Lenguaje Markdown

Markdown es un lenguaje de marcado ligero utilizado para escribir documentación de forma sencilla. GitHub interpreta automáticamente los archivos con extensión `.md`, como `README.md`, y muestra el resultado con formato.

He consultado las siguientes hojas de referencia (*Cheat Sheets*):

- [Sintaxis Markdown – Markdown.es](#)
- [Guía rápida de Markdown](#)

5.1. Modificar el archivo `README.md`

He editado el README del repositorio `prueba_sergio_mesa` con el editor de texto que prefiera. Por ejemplo, usando `nano` :

```
cd ~/github_practicas/prueba_sergio_mesa
nano README.md
```

He introducido el siguiente contenido. En este README hemos incluido todos los elementos solicitados en la práctica:

```
# Repositorio de prueba 2ASIR – Sergio Mesa
```

```
## Introducción
```

En este repositorio he realizado una práctica de **GitHub** para 2ASIR. También podemos escribir texto en *cursiva* y palabras en `código`.

```
## Ejemplo de código
```

Este es un pequeño ejemplo de código Python:

```
```python
def saludar(nombre):
 return f"Hola, {nombre}!"

print(saludar("Sergio"))
```

## Pasos realizados

---

1. He creado un repositorio en GitHub.
2. He configurado la conexión mediante SSH.
3. He subido archivos utilizando Git.

## Contenido del repositorio

---

- Documentación de la práctica
- Archivos de ejemplo
- Código básico en Python

## Enlaces

---

- Enlace externo: [GitHub](#)
- Enlace a otro fichero Markdown: [Documento adicional](#)

## Imagen

---



## Tabla de comandos Git

---

Comando	Función
<code>git clone</code>	Clona un repositorio remoto
<code>git status</code>	Muestra el estado del repositorio
<code>git add .</code>	Añade los cambios al área de preparación
<code>git commit</code>	Guarda los cambios en un commit local
<code>git push</code>	Envía los commits al repositorio remoto

```
> Edición del archivo `README.md` con sintaxis Markdown.
>
```

```

```

```
5.2. Guardar y subir el README modificado
```

Después de modificar el archivo, he guardado los cambios y he ejecutado los siguientes comandos:

```
```bash
git status
git add README.md
git commit -m "Añadido README con elementos Markdown"
git push origin main
```

Finalmente, hemos abierto el repositorio en GitHub para comprobar que el archivo `README.md` se muestra correctamente con títulos, listas, código, enlaces, imagen y tabla.

Commit y push del README modificado.

```
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$ git commit -m "Añadido README con elementos Mark
down"
[main 8341b8a] Añadido README con elementos Markdown
 1 file changed, 48 insertions(+), 2 deletions(-)
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$ git push origin main
Enumerando objetos: 5, listo.
Contando objetos: 100% (5/5), listo.
Compresión delta usando hasta 10 hilos
Comprimiendo objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (3/3), 1.02 KiB | 1.02 MiB/s, listo.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote: This repository moved. Please use the new location:
remote:   git@github.com:Sergio852/prueba_sergio_mesa.git
To github.com:sergio852/prueba_sergio_mesa.git
   afe8417..8341b8a  main -> main
sergio@Sergio-PC:~/github_practicas/prueba_sergio_mesa$
```

Repositorio de prueba 2ASIR – Sergio Mesa

Introducción

En este repositorio he realizado una práctica de **GitHub** para 2ASIR. También podemos escribir texto en *cursiva* y palabras en `código`.

Ejemplo de código

Este es un pequeño ejemplo de código Python:

```
def saludar(nombre):  
    return f"Hola, {nombre}!"  
  
print(saludar("Sergio"))
```



Pasos realizados

1. He creado un repositorio en GitHub.
2. He configurado la conexión mediante SSH.
3. He subido archivos utilizando Git.

Contenido del repositorio

- Documentación de la práctica
- Archivos de ejemplo
- Código básico en Python

Enlaces

- Enlace externo: [GitHub](#)
- Enlace a otro fichero Markdown: [Documento adicional](#)

Imagen



Tabla de comandos Git

Comando	Función
<code>git clone</code>	Clona un repositorio remoto
<code>git status</code>	Muestra el estado del repositorio
<code>git add .</code>	Añade los cambios al área de preparación
<code>git commit</code>	Guarda los cambios en un commit local
<code>git push</code>	Envía los commits al repositorio remoto